

part #1 of lecture 8:

- building blocks of distributed communication/computation.

=> collective_operation() # conceptual program interface
=> torch-distributed() # How this is implemented in NCCL/pytorch
=> benchmarking() # measure actual NCCL bandwidth.

part #2 of lecture 8: Recall MH's (FFN) are the compute bottleneck in Transformers

=> data-parallelism ^{model parallelism} => cut up along the ^{data} batch size
=> tensor-parallelism ^{data} => cut up along params width dimension
=> pipeline-parallelism => cut up along params depth dimension

Summary contents:

- => many ways to parallelize: data (batch), tensor/expert (width), pipeline (depth), sequence (length)
=> can recompute or store in memory or store in another GPU memory and communicate.
=> hardware is getting faster, but will always want bigger models
so will have this hierarchical structure.

part #3 define communication pattern across many (e.g. 256) nodes
↓
reduce/gather/scatter/broadcast/all

collective_operation()

Terms { world-size: number of device / GPUs
rank: a GPU / device

Terms { Reduce: performs some associative/commutative operation (sum, min, max)
Broadcast/scatter is a inverse/reverse of gather.
All: means destination is All devices / GPUs